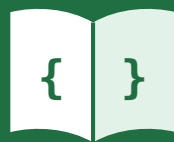

Module 15 — Observability: Metrics, Traces, Logs

Exercises

Spring Boot Edu



2026-09-24

Contents

How to Work	2
Exercise 1 — Checkout Metrics (Easy)	2
Exercise 2 — Find the Slow Step (Medium)	2
Exercise 3 — A Health Check for the Payment Provider (Medium)	3
Extra Challenge (Optional)	4

How to Work

1. The starter code is in `modules/15-observability/exercise/`. Find the `TODO` comments.
2. The exercises need no Docker: the tests use in-memory registries.
3. Run the tests:

```
./mvnw -Pexercises -pl modules/15-observability/exercise -am test
```

4. The exercise is done when all tests are green.
5. If you get stuck, read the hints first, then the solution in `modules/15-observability/solution/`.

Exercise 1 — Checkout Metrics (Easy)

Goal: the business wants to see how many checkouts happen per payment method, and how much money moves.

Tasks (`exercisel.CheckoutMetrics`):

- `TODO 1a` — a distribution summary `bookstore.checkout.amount` with the base unit `TRY`.
- `TODO 1b` — count every checkout in `bookstore.checkouts` with the tag `payment=<method>`.
- `TODO 1c` — only card and transfer are real tag values. Everything else is counted as `other`.
- `TODO 1d` — record the amount in the summary.

Hints:

- Lesson section 3.2: `Counter.builder(name).tag(key, value).register(registry).increment()`.
- `DistributionSummary.builder(name).baseUnit("TRY").register(registry),` then `record(amount.doubleValue())`.
- Why `other`? The payment method comes from the client. Without the limit, every typo would create a new time series (lesson section 4).

Acceptance criteria: all 3 tests in `ExerciselTest` pass.

Estimated time: 20 minutes

Exercise 2 — Find the Slow Step (Medium)

Goal: the monthly report takes too long, but nobody knows which step is slow. Instrument it so that a trace shows the answer.

Tasks (`exercise2.ReportService.build`):

- TODO 2a — observe the whole report as `report.build`.
- TODO 2b — observe each step as a **child** observation: `report.load-orders`, `report.load-customers`, `report.render`.

Hints:

- `Observation.createNotStarted(name, registry).observe(() -> ...)` starts, runs and stops an observation. Inside it, the observation is “current”.
- An observation started while another one is current automatically becomes its child, and in a trace, a child span.
- `observe(Supplier)` returns the supplier’s value, so the steps can pass their results on.
- The second test turns the observations into timers and checks which step is the slowest. Then start the lesson with Grafana and look at a trace to see the same thing.

Acceptance criteria: both tests in `Exercise2Test` pass.

Estimated time: 30 minutes

Exercise 3 — A Health Check for the Payment Provider (Medium)

Goal: when the payment provider is slow, down or in maintenance, the application should say so in `/actuator/health`.

Provider answers	Status	Details
in maintenance	OUT_OF_SERVICE	—
slower than 500 ms	DOWN	reason, responseTimeMs
otherwise	UP	responseTimeMs
ping() throws	DOWN	error

Tasks (`exercise3.PaymentProviderHealthIndicator.health`):

- TODO 3a — ping the provider. Maintenance → `OUT_OF_SERVICE`.
- TODO 3b — slower than 500 ms → `DOWN` with the detail `reason = "slower than 500 ms"`.
- TODO 3c — otherwise `UP` with the detail `responseTimeMs`.
- TODO 3d — an exception → `DOWN` with the error.

Hints:

- Lesson section 3.1: `Health.up().withDetail(...)`, `Health.down()`, `Health.outOfService()`.

- `Health.down(exception)` adds the exception as the detail error.
- The bean name `paymentProviderHealthIndicator` makes the component `payment-Provider` (`Exercise3ApplicationTest`).

Acceptance criteria: all tests in `Exercise3Test` and `Exercise3ApplicationTest` pass.

Estimated time: 25 minutes

Extra Challenge (Optional)

Add the `bookstore.checkouts` counter as a panel to the Grafana dashboard of the lesson (`grafana/bookstore-orders-dashboard.json`), split by payment method. Then write a Grafana alert rule that fires when there was no checkout for 10 minutes.